

Programming the FujiNet

for the Apple II

```
FINDFN    JSR    SPINIT        ; locate SmartPort
          BCS    NODEV        ; carry = no FujiNet

OPEN      LDA    #<URL        ; N:HTTPS://...
          LDX    #>URL
          JSR    NETOPEN

LOOP      JSR    NETSTAT      ; bytes waiting?
          LDA    BW
          BEQ    LOOP
          JSR    NETREAD      ; pull them in
          JSR    COUT         ; and print them
          JMP    LOOP

; the world, one SmartPort call at a time.
```

A programmer's guide and command reference for the FujiNet WiFi peripheral, from 6502 assembly language.

Free Software

FujiNet’s firmware, its client library, and this manual are free software, built and given away by a worldwide community of Apple II owners. You may copy this book for a friend — in fact, we’d be delighted. Source for everything, this booklet included, lives at `github.com/FujiNetWIFI`.

How This Book Was Verified

Every command code, parameter, payload layout, and error number in this reference was read out of the FujiNet sources, not remembered. The firmware side comes from `fujinet-firmware`: the SmartPort bus in `lib/bus/iwm/`, the device handlers in `lib/device/iwm/` (`iwmFuji.cpp`, `network.cpp`, `clock.cpp`), and the master command list in `include/fujiCommandID.h`. The host side comes from `fujinet-lib`, whose Apple II SmartPort glue lives in `apple2/apple2-6502/bus/`.

Limitation of Warranties

Neither the FujiNet community nor its contributors make any warranty with respect to this manual or to FujiNet. Everything is provided “as is.” But unlike 1984, when something bothers you, you can read the source, fix it yourself, and send a pull request.

Trademarks

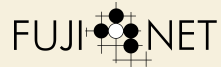
Apple, the Apple logo, Apple IIc, Apple IIGS, ProDOS, and SmartPort are trademarks of Apple Inc. Liron and UniDisk are trademarks of Apple Inc. FujiNet is a community project and is not affiliated with, endorsed by, or sponsored by Apple Inc.

Conventions

Program listings and register dumps are set in the genuine Apple II character set. Hexadecimal numbers are written with a leading dollar sign, as the Apple II has always written them: `$F9`, `$C700`. All assembly is in the syntax of the `ca65` assembler from the `cc65` suite — the same toolchain that builds `fujinet-lib` — so the examples can be assembled and linked against the library as-is.

Copyright 2026 the FujiNet contributors. Released under the GNU General Public License v3 as part of the `fujinet-manuals` repository.

Dedicated to everyone still writing 6502 by hand — and to the memory of the disk drive, which taught the Apple II to remember, and which FujiNet now teaches to dream.



Programming the FujiNet

A guide to driving the FujiNet WiFi peripheral from 6502 assembly language over SmartPort, with a complete reference to the firmware's command set — for the Apple IIc, Apple IIc Plus, Apple IIgs, and every Apple II with a SmartPort.

This book picks up where *Getting Started with FujiNet* leaves off. That book taught your fingers; this one teaches your assembler. By the last chapter you will have written a working *netcat* — open a socket, read it, write it — in a couple of pages of 6502, talking to hardware that did not exist when the 6502 did.

Contents

PREFACE	The Shape of the Thing	vii
	Two Ways In 1	
	What You Should Already Know 1	
	How the Reference Is Laid Out 2	
CHAPTER 1	The SmartPort Connection	3
	What a SmartPort Is 3	
	Finding the Dispatcher 3	
	The Shape of a Call 4	
	Four Routines You'll Use Everywhere 5	
	Reading the Result 7	
CHAPTER 2	Finding the FujiNet, Shaping a Command	9
	The Device Roll Call 9	
	The Length Header 11	
	Commands That Come in Pairs 11	
CHAPTER 3	The Network Device	13
	The Device Spec 13	
	Channels and the Device Unit 13	
	Opening and Closing 14	
	Status, Reading, and Writing 16	
	A Complete Exchange: HTTP GET 17	
	Credentials 18	
	Filesystem Operations 18	
	Reading JSON 19	
	HTTP Verbs and Headers 21	
	TCP and UDP 21	
CHAPTER 4	The Fuji Control Device	23
	The Slots Model 23	
	WiFi and the Adapter 23	
	Hosts and Device Slots 25	
	Browsing a Host 27	
	App Keys: Saving State 29	
	Boot, Devices, and Housekeeping 30	
	Hashing and QR Codes 31	
CHAPTER 5	The Clock Device	33
	Getting the Time 33	

	The Time Zone	34
APPENDIX A	Error Codes	35
	SmartPort Result Codes	35
	Network Device Status Codes	35
	Library Error Codes	35
APPENDIX B	Command Quick Reference	37
	Device Types and Calls	37
	Network Device (type \$11)	38
	Fuji Control Device (type \$10)	38
	Clock Device (type \$13)	40
APPENDIX C	netcat in 6502	41
	What It Needs	41
	The Program	41
	Trying It	43

The Shape of the Thing

A FujiNet is, electrically, a small computer of its own — an ESP32 with WiFi, a memory-card slot, and a wire that pretends to be a disk drive. The pretending is the clever part. To your Apple II, the FujiNet is not a network card and not a co-processor. It is a **SmartPort**: the same intelligent peripheral bus that an Apple IIc, a IIgs, or a Liron card uses to talk to UniDisk 3.5" drives and hard disks. Everything in this book is, underneath, a SmartPort call.

That single fact is what makes the FujiNet so easy to program. You do not install a driver. You do not patch the firmware. You find the SmartPort, you ask it which devices are attached, and you discover — alongside whatever disk drives are present — a handful of devices that were never made of metal:

- **THE_FUJI** (device type `$10`) — the control device. It mounts disk images, browses hosts, scans WiFi, reads the clock, hashes data, and keeps the slots that CONFIG shows you.
- **NETWORK** (device type `$11`) — the **N** : device. It opens TCP, UDP, HTTP, TNFS, FTP, SMB, SSH and TELNET connections and moves bytes across them, eight channels at a time.
- **FN_CLOCK** (device type `$13`) — a real-time clock with time-zone awareness.
- **CPM** (type `$12`), a **Z80** co-processor channel, and the **printer** and **modem** devices round out the family.

Two Ways In

`fujinet-lib` is the library this book quotes from. It ships prebuilt for `apple2` and `apple2enh` targets and links with `cc65`. If you are writing in C, call its functions directly and skip to the reference tables.

There are two heights at which you can program the FujiNet, and this book teaches the lower one.

The high road is `fujinet-lib`, the community client library. It gives you C functions like `network_open()` and `fuji_mount_host_slot()` and hides the bus entirely. It is the right tool for an application, and the reference chapters name the library function beside every raw command so you can find it.

The low road — the subject of this book — is to issue the SmartPort calls yourself. It costs you a page of set-up code, and buys you three things: you understand exactly what crosses the wire; you depend on nothing but the Apple's own ROM; and you can fit FujiNet support into a boot sector, a demo, or a language that has no C compiler. Every example here is raw 6502, and the last chapter — a working *netcat* — uses nothing but the routines this book builds up.

What You Should Already Know

- 6502 assembly language, and an assembler. The listings use `ca65` syntax, but they translate to Merlin or the mini-assembler without surprises.

- How a SmartPort or ProDOS block device is found in the slots — or a willingness to read Chapter 1, which derives it from scratch.
- Enough of CONFIG (from *Getting Started with FujiNet*) to know what a “host slot” and a “device slot” are.

How the Reference Is Laid Out

Chapters 1 and 2 build the foundation: how to find the FujiNet and how a single call is shaped. Chapters 3 through 5 walk the three devices you will actually program — Network, the Fuji control device, and the Clock — and every command in them is written up the same way:

EXAMPLE COMMAND

CONTROL \$F9

A one-line synopsis of what the command does, then the payload your program must build (for **CONTROL** and **WRITE**) or the layout of what comes back (for **STATUS** and **READ**):

Offset	Bytes	Meaning
0-1	2	payload length, little-endian (set by you)
2	1	an argument byte

Returns **\$00** and carry clear on success; a SmartPort error in **A** with carry set otherwise. The `fujinet-lib` equivalent is named here too.

A short listing follows each, showing the call in context. Read the next two chapters in order; after that, dip in wherever you like.

The SmartPort Connection

Before you can send the FujiNet a single command you must do two things the ROM will not do for you: find the **SmartPort dispatcher** — the entry point in some peripheral card’s firmware that accepts SmartPort calls — and learn the **unit number** the FujiNet’s devices answer to. This chapter does the first. It owes everything to the Apple SmartPort firmware specification, and the code is a direct transliteration of `fujinet-lib`’s `sp_init.s` and `sp_status_control_dispatch.s`.

What a SmartPort Is

A SmartPort is a chain of intelligent block devices hanging off one slot. Unlike the Disk II — where the Apple shovels raw bits — a SmartPort device is told what to do in **packets**: a command number and a small **parameter list** in memory. The card’s firmware does the rest. The Apple IIc and IIc Plus have a SmartPort built in; a IIGS has one per smart slot; an Apple II Plus or IIe gets one from a Liron card or any SmartPort-capable controller.

The FujiNet plugs into that chain and adds its own devices to it. To the ROM they are indistinguishable from disk drives, which is why no driver is needed. Your job is only to locate the chain and address the right link.

Finding the Dispatcher

A SmartPort card identifies itself by four bytes near the top of its `$Cn00` ROM page, where `n` is the slot number. Read low to high, slot 7 down to slot 1, and test each candidate:

Address	Value	Meaning
<code>\$Cn01</code>	<code>\$20</code>	Pascal 1.1 firmware protocol
<code>\$Cn03</code>	<code>\$00</code>	"
<code>\$Cn05</code>	<code>\$03</code>	block device
<code>\$Cn07</code>	<code>\$00</code>	SmartPort capable

When all four match, the byte at `$CnFF` is the offset to the card’s ProDOS entry point; the SmartPort entry sits **three bytes past it**. Add, and you have the address to which every call in this book is a **JSR**. Listing 1-1 finds it and patches that address straight into the dispatch routine of Listing 1-3.

The signature is the Pascal 1.1 firmware protocol with the SmartPort byte (`$00`) at offset 7. It is exactly the test the FujiNet firmware itself uses to recognise a host.

```

; --- zero page & scratch -----
PTR      =    $06          ; a 16-bit scratch pointer
;
; SPINIT: scan slots 7..1 for a SmartPort signature.
;   exit: carry CLEAR and SPVEC patched if found
;         carry SET if no SmartPort in any slot
;
SPINIT    LDX    #$C7          ; high byte of $C700 (slot 7)
SI_LOOP   STX    PTR+1
          LDA    #$00
          STA    PTR          ; PTR -> $Cn00
          LDY    #$01          ; test offsets 1,3,5,7
SI_SIG     LDA    (PTR),Y
          CMP    SIGTAB,Y
          BNE    SI_NEXT
          INY
          INY
          CPY    #$09
          BNE    SI_SIG
; --- signature matched: compute dispatch address -----
          LDY    #$FF
          LDA    (PTR),Y      ; ProDOS entry offset at $CnFF
          CLC
          ADC    #$03          ; +3 -> SmartPort entry
          STA    SPVEC        ; low byte of dispatch
          LDA    PTR+1
          ADC    #$00          ; carry into high byte
          STA    SPVEC+1      ; high byte = $Cn (+carry)
          CLC                  ; success
          RTS
SI_NEXT    DEX
          CPX    #$C0
          BNE    SI_LOOP
          SEC                  ; no SmartPort found
          RTS
; offsets 0,2,4,6 are never read, any filler will do
SIGTAB    .BYTE  $FF,$20,$FF,$00,$FF,$03,$FF,$00

```

Important!

`SPINIT` only finds the bus. It does not tell you which unit is the FujiNet — a slot may carry real disks too. Unit discovery is Chapter 2’s job, and it depends on the call machinery built next.

The Shape of a Call

Every SmartPort call is the same three-step ritual: build a parameter list in memory, **JSR** to the dispatcher, and follow the **JSR** with the command number and the address of your list — **inline, in the code stream**:

Listing 1-2. The SmartPort calling sequence

```
JSR    dispatch      ; the address SPINIT found
.BYTE  command        ; 0=STATUS 4=CONTROL 8=READ 9=WRITE
.WORD  paramlist      ; pointer to the list below
; <-- control returns HERE; A = error code, C = error
```

This is why the dispatcher is reached by **JSR** and not **JMP**: it pops the return address, reads the three bytes that follow it, then pushes a corrected address so the **RTS** lands past them.

The dispatcher reads those three bytes, then bumps its own return address past them so your program resumes on the instruction after the **\$.WORD**. On return, **A** holds the result code (**\$00** = success) and the carry flag mirrors it; **X** and **Y** hold the count of bytes transferred, low then high.

The parameter list itself is a flat block of bytes. Its first byte is the **parameter count**, which depends on the call; the rest carry the unit, a pointer to your data buffer, and either a one-byte command code (for **STATUS** and **CONTROL**) or a two-byte length (for **READ** and **WRITE**):

Offset	Bytes	Field
0	1	parameter count (3, 3, 4, 4)
1	1	unit number (the device id)
2-3	2	buffer pointer, low/high
4	1	STATUS / CONTROL code ---or---
4-5	2	READ / WRITE byte count, low/high

The four calls this book uses, with their command numbers and parameter counts, are taken verbatim from **sp.inc**:

Call	Cmd #	Pcount
Purpose	STATUS	\$00
3	ask a device for data	CONTROL
\$04	3	send a device a command + data
READ	\$08	4
read a stream of bytes	WRITE	\$09
4	write a stream of bytes	

By the Way: SmartPort also defines **OPEN** (**\$06**) and **CLOSE** (**\$07**). The FujiNet's character devices accept them but do nothing useful with them — connections are opened and closed with **CONTROL** codes instead, as Chapter 3 shows — so this book never issues them.

Four Routines You'll Use Everywhere

Rather than hand-assemble a parameter list at every call site, build it once. The routines in Listing 1-3 keep a single list (**CMDLIST**), a single 512-byte buffer (**PAYLOAD**), and a one-byte destination (**SPUNIT**) that you set before calling. They are the assembly twins of **fujinet-**

lib's `sp_status`, `sp_control`, `sp_read`, and `sp_write` every later example calls them.

Listing 1-3. The call primitives (SPSTAT, SPCTRL, SPREAD, SPWRITE)

```
PAYLOAD =    $1000          ; 512-byte data buffer (page-aligned)
;
SPUNIT      .BYTE 0         ; destination device id, set before a call
SPERR       .BYTE 0         ; result code from the last call
SPCNT       .WORD 0         ; bytes transferred (from X/Y)
CMDLIST     .RES 10         ; the parameter list
;
; SPSTAT - STATUS call.    A = status code.
SPSTAT      STA  CMDLIST+4
            LDA  #3
            STA  CMDLIST
            LDA  #$00        ; SP_CMD_STATUS
            STA  SPCMD
            JMP  SPSETUP
; SPCTRL - CONTROL call.   A = control code.
SPCTRL      STA  CMDLIST+4
            LDA  #3
            STA  CMDLIST
            LDA  #$04        ; SP_CMD_CONTROL
            STA  SPCMD
            JMP  SPSETUP
; SPREAD - READ call.      A/X = byte count (lo/hi).
SPREAD      STA  CMDLIST+4
            STX  CMDLIST+5
            LDA  #4
            STA  CMDLIST
            LDA  #$08        ; SP_CMD_READ
            STA  SPCMD
            JMP  SPSETUP
; SPWRITE - WRITE call.    A/X = byte count (lo/hi).
SPWRITE     STA  CMDLIST+4
            STX  CMDLIST+5
            LDA  #4
            STA  CMDLIST
            LDA  #$09        ; SP_CMD_WRITE
            STA  SPCMD
; --- shared: fill unit + buffer pointer, then dispatch ---
SPSETUP     LDA  SPUNIT
            STA  CMDLIST+1
            LDA  #<PAYLOAD
            STA  CMDLIST+2
            LDA  #>PAYLOAD
            STA  CMDLIST+3
            JSR  $FFFF        ; <-- operand patched by SPINIT
SPVEC      =    *-2
SPCMD       .BYTE $00        ; command number (patched above)
            .WORD CMDLIST    ; pointer to the parameter list
            STA  SPERR        ; A = result; C already reflects it
            STX  SPCNT
            STY  SPCNT+1
```

```
LDA  SPERR
RTS
```

▲ **Warning**

`SPCMD` and the `JSR` operand at `SPVEC` live in the code stream and are modified as the program runs. Keep these routines in RAM, never in ROM, and do not let an interrupt re-enter them mid-call.

Reading the Result

A call comes back with its error code in `A` and the carry flag set on any error, so the idiom is simply `BCC ok` / handle the error. The codes are SmartPort’s own (from `iwmm.h`); the ones you will actually meet are below, and Appendix A lists them all.

Code	Name	Means
\$00	NOERROR	success
\$01	BADCMD	device doesn't know that command
\$21	BADCTL	bad STATUS / CONTROL code
\$22	BADCTLPARM	bad parameter in the payload
\$27	IOERROR	the device tried and failed
\$28	NODRIVE	no such unit on the bus
\$2F	OFFLINE	device offline / no media

With Listing 1-3 assembled, a complete (if pointless) first call — ask unit 1 for its general status — is four instructions:

Listing 1-4. Your first SmartPort call

```
JSR  SPINIT          ; find the bus
BCS  NODEV
LDA  #1
STA  SPUNIT          ; talk to unit 1
LDA  #$00            ; status code 0 = device status
JSR  SPSTAT
BCS  OOPS            ; A holds the SmartPort error
; PAYLOAD now holds a 4-byte status; SPCNT = 4
```

The next chapter turns that “unit 1” guess into a real search for the FujiNet, and explains the one convention — the payload’s length header — that every Fuji and Network command obeys.

Finding the FujiNet, Shaping a Command

The bus may carry real disks as well as the FujiNet’s invented devices, so you cannot assume a unit number — you must ask. SmartPort lets you enumerate the chain and interrogate each link, and every FujiNet device announces itself with a **type byte** you can recognise.

The Device Roll Call

A **STATUS** call to **unit 0** with status code **\$00** returns the number of devices on the bus in the first payload byte. Walk the units from **1** upward and ask each for its **Device Information Block** (DIB) with status code **\$03**. The DIB is a fixed 25-byte record; the byte you want is at offset 21 — the device **type**:

Offset	Bytes	DIB field
0	1	status byte
1-3	3	block size (0 for FujiNet devices)
4	1	name length
5-20	16	device name, space-padded ("NETWORK", ...)
21	1	device TYPE <-- match on this
22	1	device subtype
23-24	2	firmware version

The FujiNet type bytes are defined in `iwm.h`. These are the constants you search for:

Type	Device	What it is
\$10	THE_FUJI	control: slots, mounts, WiFi, hashing
\$11	NETWORK	the N: device - sockets and protocols
\$12	CPM	Z80 / CP/M co-processor channel
\$13	FN_CLOCK	real-time clock
\$14	(printer)	the FujiNet printer
\$15	(modem)	the FujiNet modem

Listing 2-1 is a generic search. Hand it a type byte; it returns the unit number in **A**, or zero with carry set if no such device is on the bus. It is the spine of `fujinet-lib`’s `sp_find_device`.

Listing 2-1. Find a device by type (FINDDEV)

WANT	.BYTE 0	; the type we're hunting for
COUNT	.BYTE 0	; devices reported on the bus
IDX	.BYTE 0	; current unit under test

```

;
; FINDDEV: A = wanted type byte ($10, $11, ...).
;   exit: A = unit (1..n), carry CLEAR if found
;         A = 0, carry SET if not found
;
FINDDEV STA WANT
        LDA #0
        STA SPUNIT      ; unit 0...
        LDA #$00        ; ...status 0 = device count
        JSR SPSTAT
        BCS FD_NONE
        LDA PAYLOAD      ; count in first byte
        BEQ FD_NONE
        STA COUNT
        LDA #1
FD_LOOP STA IDX
        LDA IDX
        STA SPUNIT
        LDA #$03        ; DIB request
        JSR SPSTAT
        BCS FD_SKIP     ; no DIB for this unit
        LDA PAYLOAD+21  ; the type byte
        CMP WANT
FD_SKIP BEQ FD_FOUND
        INC IDX
        LDA IDX
        CMP COUNT
        BCC FD_LOOP
        BEQ FD_LOOP
FD_NONE LDA #0
        SEC
        RTS
FD_FOUND LDA IDX
        CLC              ; A = unit number
        RTS

```

Two thin wrappers name the devices you reach for most. Keep the results; you will pass them as **SPUNIT** before every command:

Listing 2-2. Name the two devices you'll use (FINDFUJI, FINDNET)

```

FUJIID  .BYTE 0
NETID   .BYTE 0
;
FINDFUJI LDA #$10
        JSR FINDDEV
        STA FUJIID
        RTS
FINDNET  LDA #$11
        JSR FINDDEV
        STA NETID
        RTS

```

By the Way: Historically the Fuji control device was bolted onto disk unit 0, and very old firmware may still answer there. `fujinet-lib` keeps a fallback that looks for a block device named `DISK_0` if no `#10` type is found. Current firmware exposes `THE_FUJI` properly, so a plain type search is enough.

The Length Header

Here is the one rule that governs every Fuji and Network command. When you send a device data — any `CONTROL` call, and the data half of a `WRITE` — the firmware first reads a **length**: the first two bytes of your payload, low byte then high, give the number of bytes that follow. The firmware (`iwm_decode_data_packet`) uses it to know how much to pull off the wire. So a `CONTROL` payload always looks like this:

Offset	Bytes	Meaning
0	1	length low byte
1	1	length high byte
2...	N	the command's own arguments

Important!

The length counts the bytes **after** the two-byte header — the arguments only. Forget it, or count it wrong, and the device reads garbage and returns `#22` (`BADCTLPARM`). Every `CONTROL` example in this book sets bytes 0 and 1 first.

A `STATUS` call is the mirror image: you do not send a length, you **receive** data. The device fills `PAYLOAD` with its reply and tells you the size in `ℵ/Υ` (which Listing 1-3 saved in `SPCNT`). Read it straight out of the buffer.

Commands That Come in Pairs

Because `CONTROL` sends and `STATUS` receives, several operations take **both**: a `CONTROL` to pose the question or stage the work, then a `STATUS` with the **same** code to collect the answer. Reading a directory entry, running a JSON query, and computing a hash all follow this two-step. When a reference entry shows two chips — `CONTROL $F6` `STATUS $F6` — that is what it means: send, then fetch.

With discovery and the length rule in hand, every command in the rest of the book reduces to: point `SPUNIT` at the right device, lay out the payload, and call one of the four primitives. The next three chapters are just that, command by command.

The Network Device

The Network device — type **\$11**, the one you reach as **N:** from BASIC — is where the FujiNet earns its name. Through it you open TCP and UDP sockets, fetch URLs over HTTP and HTTPS, mount remote filesystems over TNFS, FTP, and SMB, and even tunnel TELNET and SSH. All of it arrives through the same four SmartPort primitives from Chapter 1; the protocol is chosen by a **device spec** string, and the verbs are **CONTROL** codes that happen to be ASCII letters.

The Device Spec

Every network command is addressed to a string of the form

```
NCx]:PROTO: //host[:port]/path
```

The protocol is matched in the
firmware's
ProtocolParser

where **x** is the channel number 1–8, **PROTO** is one of the schemes below, and the rest is the resource. The firmware recognises:

Scheme	Use	Scheme
Use	TCP	raw TCP socket
TNFS	remote disk filesystem	UDP
datagram socket	FTP	file transfer
HTTP	web (cleartext)	SMB
Windows shares	HTTPS	web (TLS)
SSH	secure shell	TELNET
telnet		

Channels and the Device Unit

Mind the difference between two “unit” numbers. The SmartPort **device** unit — what you put in **SPUNIT** — is the Network device’s id from **FINDNET** it never changes. The **channel** (the **x** in **Nx:**, 1–8) is a logical sub-unit selected by a **CONTROL** code, **\$FA**, before you operate on it. Send it once when you switch channels; every later call acts on the selected one.

SET CHANNEL

CONTROL \$FA

Selects which **Nx:** channel subsequent network commands act on.

Offset	Bytes	Value
0-1	2	length = \$0001
2	1	channel number, 1-8

Returns \$00 on success. fujinet-lib: `network_set_unit( )`, called automatically inside every other network call.

Listing 3-1. Select a channel (NETCHAN)

```
; NETCHAN: A = channel number (1..8)
NETCHAN STA PAYLOAD+2
        LDA #1
        STA PAYLOAD          ; length low  = 1
        LDA #0
        STA PAYLOAD+1        ; length high = 0
        LDA NETID
        STA SPUNIT
        LDA #$FA              ; NETCMD_SET_CHANNEL
        JMP SPCTRL
```

Opening and Closing

OPEN

CONTROL \$4F 'O'

Instantiates the protocol named in the device spec and connects. The payload carries an access **mode** and a translation **mode**, then the spec and a terminating zero.

Offset	Bytes	Value
0-1	2	length = (spec length) + 3
2	1	access mode (see table)
3	1	translation mode (see table)
4...	N+1	device spec string, NUL-terminated

Mode	Access	\$04	Trans	Line endings	\$00
read	\$08	writ	none (binary)	\$01	CR
\$0C	read/write	\$0D	\$02	LF	\$03
HTTP POST	\$05	HTTP	CR/LF	\$04	PETSCII

Returns \$00 on success; \$22 if the spec is malformed. Check the device error with a `STATUS` call afterwards. fujinet-lib: `network_open( )`.

Listing 3-2. Open a connection (NETOPEN, COPYSPEC)

```
URLPTR = $08          ; zp pointer -> device spec
MODE   .BYTE 0
TRANS  .BYTE 0
;
; NETOPEN: URLPTR -> NUL-terminated spec; MODE, TRANS set.
;          select the channel first with NETCHAN.
NETOPEN JSR COPYSPEC    ; copy spec to PAYLOAD+4, Y = length
```

```

        TYA
        CLC
        ADC    #3                ; +mode +trans +NUL
        STA    PAYLOAD           ; length low
        LDA    #0
        ADC    #0
        STA    PAYLOAD+1        ; length high (spec may exceed 252)
        LDA    MODE
        STA    PAYLOAD+2
        LDA    TRANS
        STA    PAYLOAD+3
        LDA    NETID
        STA    SPUNIT
        LDA    #'0'             ; NETCMD_OPEN
        JMP    SPCTRL
;
; COPYSPEC: copy the spec (with its NUL) to PAYLOAD+4.
;           exit Y = string length (offset of the NUL).
COPYSPEC LDY    #0
CS_LOOP  LDA    (URLPTR),Y
        STA    PAYLOAD+4,Y
        BEQ    CS_DONE          ; the NUL is now copied
        INY
        BNE    CS_LOOP
CS_DONE  RTS

```

CLOSE

CONTROL \$43 'C'

Closes the channel, flushes and frees its buffers. The spec identifies the channel; only its unit digit really matters.

Offset	Bytes	Value
0-1	2	length = (spec length) + 1
2...	N+1	device spec string, NUL-terminated

Returns \$00 on success. fujinet-lib: `network_close()`.

Listing 3-3. Close a connection (NETCLOSE)

```

NETCLOSE LDY    #0
NC_LOOP  LDA    (URLPTR),Y
        STA    PAYLOAD+2,Y
        BEQ    NC_DONE
        INY
        BNE    NC_LOOP
NC_DONE  INY          ; count the NUL
        STY    PAYLOAD ; length low
        LDA    #0
        STA    PAYLOAD+1 ; length high
        LDA    NETID
        STA    SPUNIT

```

```
LDA #'C'          ; NETCMD_CLOSE
JMP SPCTRL
```

Status, Reading, and Writing

These three are the working day of the Network device. **STATUS** tells you how many bytes have arrived and whether the far end is still connected; **READ** drains them; **WRITE** sends.

STATUS

STATUS \$53 'S'

Returns the channel's pending byte count, connection flag, and the device-specific error code.

Offset	Bytes	Returned value
0-1	2	bytes waiting to be read
2	1	connected: 1 = open, 0 = far end closed
3	1	device error (1 = OK, 136 = EOF)

Returns Byte count also left in **SPCNT**. fujinet-lib: `network_status( )`.

Listing 3-4. Poll a channel (NETSTAT)

BW	.WORD 0	; bytes waiting
CONN	.BYTE 0	; connected flag
NERR	.BYTE 0	; device error
;		
NETSTAT	LDA NETID	
	STA SPUNIT	
	LDA #'S'	; NETCMD_STATUS
	JSR SPSTAT	
	BCS NS_X	
	LDA PAYLOAD	
	STA BW	
	LDA PAYLOAD+1	
	STA BW+1	
	LDA PAYLOAD+2	
	STA CONN	
	LDA PAYLOAD+3	
	STA NERR	
NS_X	RTS	

READ

READ (cmd \$08)

Reads up to the requested number of bytes — never more than are waiting, and never more than 512 in one call — into **PAYLOAD**. Always poll **STATUS** first so you ask only for what is there.

Returns Data in **PAYLOAD** actual count in **SPCNT**. fujinet-lib: `network_read( )` (which loops this for you).

Listing 3-5. Read what's waiting (NETREAD)

```
; NETREAD: read BW bytes (caller guarantees BW <= 512) into PAYLOAD
NETREAD LDA NETID
        STA SPUNIT
        LDA BW
        LDX BW+1
        JSR SPREAD          ; A/X = byte count
        RTS                ; data in PAYLOAD, SPCNT = bytes read
```

WRITE

WRITE (cmd \$09)

Sends bytes from **PAYLOAD** to the channel. Write at most 512 per call; loop for more.

Returns **\$00** on success. fujinet-lib: **network_write()**.

Listing 3-6. Write a buffer (NETWRITE)

```
; NETWRITE: A/X = byte count (lo/hi); data already in PAYLOAD
NETWRITE LDY NETID
        STY SPUNIT
        JMP SPWRITE          ; A/X preserved; returns A = error
```

A Complete Exchange: HTTP GET

The five routines above are enough to fetch a web page. Select a channel, open the URL for reading, then poll-and-read until the far end signals EOF (device error **136**). Listing 3-7 prints a URL to the 40-column screen through the monitor's **COUT**.

Listing 3-7. Fetch a URL and print it

```
COUT    =    $FDED          ; monitor character out
;
GET      JSR SPINIT
        BCS NODEV
        JSR FINDNET        ; -> NETID
        LDA NETID
        BEQ NODEV
        LDA #1
        JSR NETCHAN        ; use N1:
        LDA #<URL
        STA URLPTR
        LDA #>URL
        STA URLPTR+1
        LDA #$04           ; mode = read
        STA MODE
        LDA #$00           ; trans = none
```

```

        STA  TRANS
        JSR  NETOPEN
        BCS  DONE
GLOOP   JSR  NETSTAT
        LDA  NERR
        CMP  #136           ; EOF?
        BEQ  DONE
        LDA  BW
        ORA  BW+1
        BEQ  GLOOP          ; nothing yet, poll again
        ; clamp BW to 512 if larger (omitted for brevity)
        JSR  NETREAD
        LDX  #0
PLOOP   LDA  PAYLOAD,X
        ORA  #$80           ; high bit set for the Apple video
        JSR  COUT
        INX
        CPX  SPCNT
        BNE  PLOOP
        JMP  GLOOP
DONE    JSR  NETCLOSE
        RTS
URL     .BYTE "N1:HTTP://FUJINET.ONLINE/", 0
```

▲ **Warning**

READ returns at most 512 bytes, so clamp **BW** to 512 before **NETREAD** when a channel has more than that waiting. The library’s **network_read** does this; the netcat in Appendix C shows the few extra instructions.

Credentials

For schemes that authenticate — FTP, SMB — set a username and password **before** **OPEN**. Both are **CONTROL** codes carrying a fixed 256-byte string field.

USERNAME / PASSWORD

CONTROL \$FD / \$FE

Offset	Bytes	Value
0-1	2	length = \$0100 (256)
2...	256	credential string (NUL-padded)

Returns **\$00** on success. Username is **\$FD**, password **\$FE**. No library wrapper on the Apple II — issue them raw.

Filesystem Operations

When a channel speaks a filesystem protocol (TNFS, FTP, SMB, even HTTP with WebDAV), six **CONTROL** codes manage files and directories. Each takes the same payload — a device spec naming the target — so one example covers them all.

Code	Char	Operation
fujinet-lib	\$21	!
delete file	network_fs_delete	\$20
(sp)	rename (spec is from,to)	network_fs_rename
\$23	#	lock (make read-only)
network_fs_lock	\$24	\$
unlock	network_fs_unlock	\$2A
*	make directory	network_fs_mkdir
\$2B	+	remove directory
network_fs_rmdir	\$2C	,
change directory	network_fs_cd	

Listing 3-8. Delete a remote file

```
; delete the file named by URLPTR's spec, e.g.
; "N1:TNFS://TMA-2/OLD.TXT"
RM      LDY    #0
RM_LP   LDA    (URLPTR),Y
        STA    PAYLOAD+2,Y
        BEQ    RM_END
        INY
        BNE    RM_LP
RM_END  INY          ; include NUL
        STY    PAYLOAD      ; length low
        LDA    #0
        STA    PAYLOAD+1
        LDA    NETID
        STA    SPUNIT
        LDA    #'!'          ; NETCMD_DELETE
        JMP    SPCTRL
```

By the Way: GETCWD (STATUS \$30, the digit 0) returns the current directory prefix in the payload — the read-side companion to CHDIR.

Reading JSON

The Network device can parse a JSON document on the FujiNet and hand you single fields, so a 1 MHz 6502 never has to. Open the resource, switch the channel into **JSON mode**, parse, then query a path as many times as you like; each query's result is fetched with STATUS + READ exactly as a normal read.

CHANNEL MODE

CONTROL SFC

Switches the channel between protocol mode (0, the default) and JSON mode (1).

Offset	Bytes	Value
0-1	2	length = \$0001
2	1	0 = protocol, 1 = JSON

JSON PARSE

CONTROL \$50 'P'

Parses the document currently waiting on the channel. No arguments (length \$0000). fujinet-lib pairs this with the mode switch above as `network_json_parse()`.

JSON QUERY

CONTROL \$51 'Q'

Sets the JSONPath to read; the value is then retrieved with a normal `STATUS/READ`.

Offset	Bytes	Value
0-1	2	length = (query length) + 1
2...	N+1	JSONPath string, e.g. "/weather/temp", NUL

Returns After the query, `STATUS` reports the value length; a `READ` of that many bytes returns the value. fujinet-lib: `network_json_query()`.

Listing 3-9. Query one field after an open

; assumes the resource is open on the selected channel		
JPARSE	LDA	#1
	STA	PAYLOAD ; len = 1
	LDA	#0
	STA	PAYLOAD+1
	LDA	#1
	STA	PAYLOAD+2 ; mode = JSON
	LDA	NETID
	STA	SPUNIT
	LDA	##FC ; CHANNEL_MODE
	JSR	SPCTRL
	LDA	#0
	STA	PAYLOAD ; len = 0
	STA	PAYLOAD+1
	LDA	#'P' ; PARSE
	JSR	SPCTRL
	; --- now ask for a field ---	
JQ_LP	LDY	#0
	LDA	QUERY,Y
	STA	PAYLOAD+2,Y
	BEQ	JQ_END
	INY	
	BNE	JQ_LP
JQ_END	INY	; include NUL
	STY	PAYLOAD
	LDA	#0
	STA	PAYLOAD+1
	LDA	#'Q' ; QUERY
	JSR	SPCTRL

```
JSR NETSTAT      ; BW = value length
JSR NETREAD      ; value now in PAYLOAD
RTS
QUERY .BYTE "/weather/0/main", 0
```

HTTP Verbs and Headers

An **HTTP/HTTPS** channel is more than a byte pipe. The access mode chosen at **OPEN** selects the verb — read is **GET**, **\$00** is **POST**, **\$05** is **DELETE** — and one **CONTROL** code, **\$4D**, steers the channel between the request body and its headers.

HTTP CHANNEL MODE

CONTROL \$4D 'M'

Directs reads and writes on an HTTP channel to the body or the headers. The mode travels in the **second** argument byte.

Offset	Bytes	Value
0-1	2	length = \$0002
2	1	0
3	1	channel mode (see table)

Mode	Meaning	Mode	Meaning	3
body (default)	1	set request headers	4	POST: set data
2	read response headers			

Returns **\$00** on success. `fujinet-lib: network_http_set_channel_mode( )` a header is then sent with an ordinary `WRITE (network_http_add_header( )`.

TCP and UDP

A raw **TCP :** channel opened in read/write mode is a bidirectional socket — the foundation of the netcat in Appendix C. Two extra **CONTROL** codes serve the listening and datagram cases.

TCP ACCEPT / CLOSE CLIENT

CONTROL \$41 / \$63

On a **TCP :** channel opened to listen, **\$41** (**'A'**) accepts a waiting client; **\$63** (**'C'**) closes the current client while keeping the listener alive. Both take an empty payload (length **\$0000**).

UDP SET DESTINATION

CONTROL \$44 'D'

For a **UDP :** channel, sets the host:port that the next **WRITE** datagrams are addressed to.

Offset	Bytes	Value
0-1	2	length = (string length)
2...	N	"host:port" string

Returns Companion `$72 ('r', GET_REMOTE)` reports the address of the last datagram's sender into the payload — handy for replying.

That is the whole Network device. Point `SPUNIT` at `NETID`, select a channel, and these codes give you the Internet. The next chapter turns to the other half of the FujiNet: the control device that manages disks, hosts, and the hardware itself.

The Fuji Control Device

The device named **THE_FUJI** — type **\$10** — is the one CONFIG talks to. It owns the WiFi radio, the list of **hosts** (TNFS, SMB and the like) and **disk image** mounts, the directory browser, persistent app-key storage, and a drawer of utilities. Where the Network device overloaded ASCII letters as verbs, the Fuji device uses the high-numbered **FUJICMD_** codes from `fujiCommandID.h`. Set **SPUNIT** to **FUJI ID** (from **FINDFUJI**); the rule from Chapter 2 holds — **STATUS** receives, **CONTROL** sends, and the length header leads every **CONTROL** payload.

Not every code in `fujiCommandID.h` is serviced on the Apple II. This chapter documents those the firmware's `iwmfuji.cpp` actually dispatches; Appendix B's table marks every code's status.

The Slots Model

CONFIG presents two arrays you will meet constantly. **Host slots** (8 of them) name the places disks live — a TNFS server, an SMB share, the SD card. **Device slots** are the drive bays: each remembers a host, an access mode, and a filename, and maps to a SmartPort disk unit. Mounting is a two-step you will recognise from CONFIG: mount a host, browse it, then mount one of its images into a device slot.

WiFi and the Adapter

SCAN NETWORKS

STATUS \$FD

Triggers a scan and returns the count of access points found, in the first payload byte. fujinet-lib: `fuji_scan_for_networks()`.

GET SCAN RESULT

CONTROL \$FC + STATUS \$FC

A paired command. **CONTROL** with the index selects an access point; the following **STATUS** returns its name and signal.

Phase	Offset	Bytes
Meaning	CONTROL	0-1 / 2
2 / 1	length \$0001; AP index n	STATUS
0-32 / 33	33 / 1	SSID (NUL-padded); RSSI (signed)

Returns fujinet-lib: `fuji_get_scan_result()`.

GET / SET SSID

STATUS \$FE / CONTROL \$FB

STATUS \$FE returns the stored network config — a 33-byte SSID followed by a 64-byte password. **CONTROL** \$FB sets the same structure (length = its size), joining the network. fujinet-lib: `fuji_get_ssid()` / `fuji_set_ssid()`.

GET WIFI STATUS

STATUS \$FA

Returns one byte: 3 = connected, 6 = disconnected. **STATUS** \$EA (GET WIFI ENABLED) returns 1 if the radio is on. fujinet-lib: `fuji_get_wifi_status()`.

GET ADAPTER CONFIG

STATUS \$E8

Returns the live network configuration in one shot: the joined SSID, hostname, and the four-byte IP, gateway, netmask, DNS, plus MAC and BSSID and a firmware version string.

Offset	Bytes	Field
0	33	SSID
33	64	hostname
97	4	local IP
101	4	gateway
105	4	netmask
109	4	DNS IP
113	6	MAC address
119	6	BSSID
125	15	firmware version string

Returns STATUS \$C4 returns the same plus ready-made string forms of every address. fujinet-lib: `fuji_get_adapter_config()` / `_extended()`.

Listing 4-1. Print the FujiNet's IP address

PR TIP	JSR	FINDFUJI	; -> FUJIID
	LDA	FUJIID	
	STA	SPUNIT	
	LDA	#\$E8	; GET ADAPTERCONFIG
	JSR	SPSTAT	
	BCS	IPERR	
	LDX	#0	; print local IP as N.N.N.N
DOTLP	LDA	PAYLOAD+97,X	; localIP starts at offset 97
	JSR	PRBYTE	; monitor: print A in decimal/hex
	CPX	#3	
	BEQ	IPDONE	
	LDA	#'.' + \$80	
	JSR	COUT	
	INX		

IPDONE	BNE RTS	DOTLP
--------	------------	-------

Hosts and Device Slots

READ / WRITE HOST SLOTS

STATUS \$F4 / CONTROL \$F3

The eight host slots are an array of eight 32-byte names. **STATUS \$F4** reads all 256 bytes; **CONTROL \$F3** writes them back (length = 256). fujinet-lib: `fuji_get_host_slots()` / `fuji_put_host_slots()`.

READ / WRITE DEVICE SLOTS

STATUS \$F2 / CONTROL \$F1

Device slots are an array of 38-byte records:

Offset	Bytes	Field
0	1	host slot this disk lives on
1	1	access mode (1 = read, 2 = read/write)
2	36	filename

Returns **STATUS \$F2** returns the whole array; **CONTROL \$F1** writes it (length = 38 × count). Library calls: get and put device slots.

MOUNT / UNMOUNT HOST

CONTROL \$F9 / \$E6

Brings a host slot online (connects the server) or takes it offline. The payload is one byte: the host slot number.

Offset	Bytes	Value
0-1	2	length = \$0001
2	1	host slot number

Returns fujinet-lib: `fuji_mount_host_slot()` / `fuji_unmount_host_slot()`.

MOUNT / UNMOUNT IMAGE

CONTROL \$F8 / \$E9

Mounts the disk image recorded in a **device** slot, making it a live SmartPort disk. **MOUNT** takes the slot and an access mode; **UNMOUNT** takes just the slot.

Offset	Bytes	Value
0-1	2	length (\$0002 mount, \$0001 unmount)
2	1	device slot number
3	1	access mode (mount only: 1=RO, 2=RW)

Returns fujinet-lib: fuji_mount_disk_image() / fuji_unmount_disk_image().CONTROL \$D7 (MOUNT ALL, empty payload) mounts every configured slot at once.

Listing 4-2. Mount host 0, then image in device slot 1

```
MOUNT JSR FINDFUJI
      LDA FUJIID
      STA SPUNIT
      LDA #1
      STA PAYLOAD      ; length = 1
      LDA #0
      STA PAYLOAD+1
      STA PAYLOAD+2    ; host slot 0
      LDA #$F9         ; MOUNT HOST
      JSR SPCTRL
      BCS MERR
      LDA #2
      STA PAYLOAD      ; length = 2
      LDA #0
      STA PAYLOAD+1
      LDA #1
      STA PAYLOAD+2    ; device slot 1
      LDA #2
      STA PAYLOAD+3    ; mode = read/write
      LDA #$F8         ; MOUNT IMAGE
      JMP SPCTRL
```

SET / GET DEVICE FULLPATH

CONTROL \$E2 / STATUS \$A0+ds

CONTROL \$E2 records a filename into a device slot without mounting; payload is device slot, host slot, mode, then the filename and a NUL. To read a slot's filename back, issue STATUS with code \$A0 + ds (so \$A0...\$A9 for slots 0–9); the payload returns the 256-byte path.

Offset	Bytes	Value (SET)
0-1	2	length = (name length) + 4
2	1	device slot
3	1	host slot
4	1	access mode
5...	N+1	filename, NUL-terminated

Returns fujinet-lib: fuji_set_device_filename() / fuji_get_device_filename().

NEW DISK

CONTROL \$E7

Creates a fresh blank image on a host and records it in a device slot.

Offset	Bytes	Value
0-1	2	length = sizeof(NewDisk)
2	1	host slot
3	1	device slot
4	1	create/media type
5-8	4	block count (little-endian)
9...	256	filename

Returns fujinet-lib: `fuji_create_new()`.

Browsing a Host

To list files on a mounted host, open its directory, read entries one at a time until the end marker, then close.

OPEN DIRECTORY

CONTROL \$F7

Opens a directory on a host slot. The path and an optional filename filter are packed into one field, separated by a NUL.

Offset	Bytes	Value
0	1	length low = \$00
1	1	length high = \$01 (a full 256-byte field)
2	1	host slot
3...	255	path, NUL, optional filter

Returns fujinet-lib: `fuji_open_directory()`.

READ DIR ENTRY

CONTROL \$F6 + STATUS \$F6

A paired command. **CONTROL** sets the maximum length to return and a flags byte; the following **STATUS** returns one entry. Set bit 7 of the flags to append a details block after the name.

Phase	Offset	Bytes
Meaning	CONTROL	2 / 3
1 / 1	max length; flags (\$80 = + details)	STATUS
0...	maxlen	filename; details if requested

The optional details block, appended after the filename, is:

Offset	Bytes	Detail
0-5	6	modified: year, month, day, hour, min, sec
6-9	4	file size (little-endian)
10	1	is-directory flag
11	1	name-was-truncated flag
12	1	media type

Important!

A returned entry whose first byte is **\$7F** is the **end-of-directory** marker — stop reading. On the Apple II the firmware also prepends two spaces (**\$20**) to each real filename; skip them.

Returns fujinet-lib: `fuji_read_directory()`.

CLOSE DIRECTORY

CONTROL \$F5

Closes the open directory. Empty payload (length **\$0000**). **STATUS \$E5** / **CONTROL \$E4** get and set the directory read position for paging. fujinet-lib: `fuji_close_directory()`.

Listing 4-3. List a directory to the screen

```
; host slot 0 already mounted; FUJIID set
LISTDIR LDA FUJIID
        STA SPUNIT
        LDA #0           ; length = $0100 (256-byte field)
        STA PAYLOAD
        LDA #1
        STA PAYLOAD+1
        LDA #0
        STA PAYLOAD+2
        LDA #'/'         ; host slot 0
                        ; path = "/"
        STA PAYLOAD+3
        LDA #0
        STA PAYLOAD+4
        LDA ##F7         ; NUL: no filter
                        ; OPEN DIRECTORY
        JSR SPCTRL
LD_NEXT LDA #2
        STA PAYLOAD      ; length = 2
        LDA #0
        STA PAYLOAD+1
        LDA #40
        STA PAYLOAD+2    ; max 40 chars
        LDA #0
        STA PAYLOAD+3
        LDA ##F6         ; name only, no details
                        ; READ DIR ENTRY (control)
        JSR SPCTRL
        LDA ##F6         ; READ DIR ENTRY (status)
        JSR SPSTAT
        LDA PAYLOAD
        CMP ##7F         ; end-of-directory?
        BEQ LD_END
        LDX #2           ; skip the two pad spaces
```

```
PR_LP    LDA    PAYLOAD,X
        BEQ    PR_EOL
        ORA    #$80
        JSR    COUT
        INX
PR_EOL   BNE    PR_LP
        LDA    #$80          ; carriage return
        JSR    COUT
        JMP    LD_NEXT
LD_END   LDA    #0
        STA    PAYLOAD
        STA    PAYLOAD+1    ; length = 0
        LDA    #$F5          ; CLOSE DIRECTORY
        JMP    SPCTRL
```

App Keys: Saving State

An **app key** is a small block (up to 64 bytes) the FujiNet stores for your program, indexed by a creator id, an app id, and a key id — handy for high scores, settings, or a save game. You always **OPEN** the key first, declaring read or write, then **READ** or **WRITE** it.

OPEN APPKEY

CONTROL \$DC

Offset	Bytes	Value
0-1	2	length = \$0006
2-3	2	creator id (little-endian)
4	1	app id
5	1	key id
6	1	mode: 0 = read, 1 = write
7	1	reserved (\$00)

READ / WRITE APPKEY

STATUS \$DD / CONTROL \$DE

After an **OPEN** in read mode, **STATUS \$DD** returns the key's bytes (count in **SPCNT**). After an **OPEN** in write mode, **CONTROL \$DE** stores them: payload is the byte count then the data.
Returns `fujinet-lib: fuji_read_appkey( ) / fuji_write_appkey( )` (which wrap the open for you).

Listing 4-4. Save 16 bytes to app key 1

```
SAVEKEY  LDA    FUJIID
        STA    SPUNIT
        LDA    #6
```

```

        STA  PAYLOAD          ; length = 6
        LDA  #0
        STA  PAYLOAD+1
        LDA  #<CREATOR
        STA  PAYLOAD+2
        LDA  #>CREATOR
        STA  PAYLOAD+3
        LDA  #APPID
        STA  PAYLOAD+4
        LDA  #1
        STA  PAYLOAD+5        ; key id 1
        LDA  #1
        STA  PAYLOAD+6        ; mode = write
        LDA  #0
        STA  PAYLOAD+7
        LDA  #$DC              ; OPEN APPKEY
        JSR  SPCTRL
        BCS  SK_ERR
        LDA  #16
        STA  PAYLOAD          ; length = 16
        LDA  #0
        STA  PAYLOAD+1
        LDX  #0                ; copy 16 bytes of state in
SK_CP   LDA  STATE,X
        STA  PAYLOAD+2,X
        INX
        CPX  #16
        BNE  SK_CP
        LDA  #$DE              ; WRITE APPKEY
        JMP  SPCTRL
CREATOR = $1234
APPID   = 1

```

Boot, Devices, and Housekeeping

A handful of one-byte and empty-payload commands round out the device.

Code	Call	Action
Payload	\$D9	CONTROL
enable/disable CONFIG boot	1 byte: toggle	\$D6
CONTROL	set boot mode (e.g. Lobby)	1 byte: mode
\$D5	CONTROL	enable a device
1 byte: device id	\$D4	CONTROL
disable a device	1 byte: device id	\$D1
STATUS	device-enabled status	returns 1 byte
\$D7	CONTROL	mount all slots
empty	\$D8	CONTROL
copy file between hosts	src,dst,spec	\$BB
STATUS	generate a GUID string	returns 37 bytes
\$C1	STATUS	free heap (debug)
returns 4 bytes	\$53	STATUS
Fuji status block	returns 4 bytes	\$FF
CONTROL	reset the FujiNet	empty

Returns fujinet-lib names follow the obvious pattern — `fuji_set_boot_mode()`, `fuji_enable_device()`, `fuji_generate_guid()`, `fuji_reset()`, and so on.

Hashing and QR Codes

The firmware can compute MD5, SHA-1, SHA-256 and SHA-512 digests and render QR codes — useful for content addressing or showing a link on screen. These live in `iwmFuji.cpp` and are driven raw; the Apple II library wrappers are presently stubs, so issue the codes yourself.

Code	Call	Step
\$C8	CONTROL	add input data to the hash buffer
\$C7	CONTROL	compute (algorithm in byte 0); clears buffer
\$C3	CONTROL	compute without clearing the buffer
\$C6	STATUS	length of the resulting digest
\$C5	STATUS	read the digest (set hex/binary via CONTROL \$C5)
\$C2	CONTROL	clear the hash buffer
\$BC	CONTROL	add input to the QR encoder
\$BD	CONTROL	encode (version, ECC, shorten in bytes 0-2)
\$BE	STATUS	length of the encoded QR data
\$BF	STATUS	read the encoded QR modules

```

; algorithm: 0=MD5 1=SHA1 2=SHA256 3=SHA512
HASHBUF LDA FUJIID
          STA SPUNIT
          ; --- feed the data ---
          LDA #<DLEN
          STA PAYLOAD          ; length = data length
          LDA #>DLEN
          STA PAYLOAD+1
          ; ...copy DLEN bytes of data to PAYLOAD+2...
          LDA #$C8             ; HASH INPUT
          JSR SPCTRL
          ; --- compute MD5 ---
          LDA #1
          STA PAYLOAD          ; length = 1
          LDA #0
          STA PAYLOAD+1
          STA PAYLOAD+2        ; 0 = MD5
          LDA #$C7             ; HASH COMPUTE
          JSR SPCTRL
          ; --- read the 16-byte digest ---
          LDA #$C5             ; HASH OUTPUT
          JSR SPSTAT           ; digest now in PAYLOAD, SPCNT = 16
          RTS

```

That is the Fuji control device. Between it and the Network device you can do anything CONFIG can, and a good deal it cannot. One small device remains — the clock — and then we build the netcat.

The Clock Device

An Apple II has never known what time it is. The FujiNet does — it keeps network time and a configured time zone — and exposes it as device type **\$13**, **FN_CLOCK**. Find it with **FINDDEV** and the type byte **\$13** then a single **STATUS** call returns the time in whichever of six formats you ask for. The status **code** you send is an ASCII letter naming the format.

Getting the Time

GET TIME

STATUS (format letter)

The reply lands in **PAYLOAD** its length is in **SPCNT**. Binary formats are raw numbers; string formats are NUL-terminated ISO text. Send the upper-case letter to use the FujiNet’s own time zone.

Code	Char	Format returned
\$54	T	simple binary: cent, yr, mon, day, hr, min, sec
\$50	P	ProDOS date/time (4 bytes)
\$41	A	APETIME binary (6 bytes)
\$49	I	ISO 8601 string, local zone
\$5A	Z	ISO 8601 string, UTC
\$53	S	Apple /// SOS format

Returns fujinet-lib: **clock_get_time()**, with format constants **SIMPLE_BINARY**, **PRODOS_BINARY**, **TZ_ISO_STRING**, **UTC_ISO_STRING**, and so on.

The seven bytes of the simple binary format are the easiest to act on — no parsing, just numbers. The century byte is **20** for the twenty-first; add the year byte for the full year.

Listing 5-1. Read the time into seven bytes

```
NOW      .RES 7           ; cent, year, month, day, hour, min, sec
;
GETTIME  LDA  ##13
        JSR  FINDDEV      ; locate FN_CLOCK
        BCS  NOCLOCK
        STA  SPUNIT
        LDA  #'T'         ; simple binary, FujiNet time zone
        JSR  SPSTAT
        BCS  NOCLOCK
        LDX  #0           ; copy the seven bytes out
        LDA  PAYLOAD,X
        STA  NOW,X
```

```
INX
CPX #7
BNE GT_CP
RTS
```

By the Way: Send the **lower-case** letter ('t', 'i', ...) and the clock answers using an **alternate** time zone instead of the FujiNet's — the one you last set with **CONTROL \$74**. This is how `clock_get_time_tz()` fetches another city's time without disturbing the system zone.

The Time Zone

GET TIME ZONE

STATUS \$47 'G'

Returns the configured POSIX time-zone string (for example `CST6CDT,M3.2.0,M11.1.0`) into **PAYLOAD**, NUL-terminated. `fujinet-lib: clock_get_tz()`.

SET TIME ZONE

CONTROL \$54 / \$74

Sets the time zone from a string. **CONTROL \$54** ('T') sets the FujiNet's own zone and saves it; **CONTROL \$74** ('t') sets the temporary alternate zone used by the lower-case time formats.

Offset	Bytes	Value
0-1	2	length = (string length) + 1
2...	N+1	POSIX TZ string, NUL-terminated

Returns `fujinet-lib: clock_set_tz()`.

With the clock read, every device the Apple II programmer can reach through the FujiNet has been covered. What remains is to put the Network device through its paces — a real program, start to finish.

Error Codes

Two layers of error codes meet in FujiNet programming. The SmartPort layer returns the code in `A` (and the carry flag) from every call; these are Apple's own, defined in `iwmm.h`. The network **device** reports a second, finer code in the fourth byte of its `STATUS` reply — most importantly `136`, end-of-file.

SmartPort Result Codes

Code	Name	Meaning
\$00	NOERROR	success
\$01	BADCMD	device does not support the command
\$06	BUSERR	communications error on the bus
\$21	BADCTL	invalid STATUS or CONTROL code
\$22	BADCTLPARM	invalid parameter in the payload
\$27	IOERROR	the device tried the operation and failed
\$28	NODRIVE	no device at that unit number
\$2B	NOWRITE	media is write-protected
\$2D	BADBLOCK	invalid block number
\$2E	DISKSW	media was swapped (extended calls)
\$2F	OFFLINE	device offline or no media present
\$30	BADWIFI	error joining the requested SSID

Network Device Status Codes

Read from byte 3 of a Network `STATUS` reply (Listing 3-4's `NERR`):

Code	Meaning	1
normal - connected, no error	136	end of file - the resource is fully read

Library Error Codes

When you call `fujinet-lib` instead of issuing raw calls, it folds the SmartPort codes into a small device-agnostic set (`fn_error()`):

Code	Name	Meaning
\$00	FN_ERR_OK	no error
\$01	FN_ERR_IO_ERROR	an I/O problem with the device
\$02	FN_ERR_BAD_CMD	called with bad arguments
\$03	FN_ERR_OFFLINE	the device is offline
\$04	FN_ERR_WARNING	non-fatal device warning
\$05	FN_ERR_NO_DEVICE	no network device present
\$FF	FN_ERR_UNKNOWN	an unmapped device error

Command Quick Reference

Everything in this book, condensed. Codes are the bytes you place in the parameter list — as a STATUS/CONTROL code, or as the command number for READ/WRITE.

Device Types and Calls

Type	Device	Cmd	SmartPort call	\$00
THE_FUJI (control)	\$11	STATUS	\$04	CONTROL
\$12	CPM	\$106	OPEN	\$07
FN_CLOCK	\$14	CLOSE	\$08	READ
\$15	modem	\$09	WRITE	

Network Device (type \$11)

Code	Call	Operation
\$FA	CONTROL	set channel (select Nx unit)
\$4F 'O'	CONTROL	open connection (mode, trans, spec)
\$43 'C'	CONTROL	close connection
\$53 'S'	STATUS	channel status (bw, conn, err)
-	READ	read waiting bytes
-	WRITE	write bytes
\$FD / \$FE	CONTROL	set username / password
\$FC	CONTROL	channel mode (0 protocol, 1 JSON)
\$50 'P'	CONTROL	JSON parse
\$51 'Q'	CONTROL	JSON query (then STATUS+READ)
\$2C ', '	CONTROL	change directory
\$30 '0'	STATUS	get current directory
\$20	CONTROL	rename (spec is from,to)
\$21 '!'	CONTROL	delete file
\$23 '#'	CONTROL	lock file
\$24 '\$'	CONTROL	unlock file
\$2A '*'	CONTROL	make directory
\$2B '+'	CONTROL	remove directory
\$4D 'M'	CONTROL	HTTP channel mode (body/headers)
\$41 'A'	CONTROL	TCP accept connection
\$63 'c'	CONTROL	TCP close client
\$44 'D'	CONTROL	UDP set destination
\$72 'r'	CONTROL	UDP get remote address

Fuji Control Device (type \$10)

The commands the Apple II firmware services, by phase. Codes not listed here exist in `fujiCommandID.h` but are not dispatched on the Apple II.

Code	Call	Operation
\$FF	CONTROL	reset FujiNet
\$FE / \$FB	STATUS / CTRL	get / set SSID
\$FD	STATUS	scan networks (returns count)
\$FC	CTRL+STAT	get scan result n
\$FA	STATUS	get WiFi status
\$EA	STATUS	get WiFi enabled
\$F9 / \$E6	CONTROL	mount / unmount host slot
\$F8 / \$E9	CONTROL	mount / unmount disk image
\$D7	CONTROL	mount all
\$F4 / \$F3	STATUS / CTRL	read / write host slots
\$F2 / \$F1	STATUS / CTRL	read / write device slots
\$E2	CONTROL	set device filename
\$A0-\$A9	STATUS	get device filename (slot 0-9)
\$E7	CONTROL	new (blank) disk
\$F7	CONTROL	open directory
\$F6	CTRL+STAT	read directory entry
\$F5	CONTROL	close directory
\$E5 / \$E4	STATUS / CTRL	get / set directory position
\$E8 / \$C4	STATUS	adapter config / extended
\$DC	CONTROL	open app key
\$DD / \$DE	STATUS / CTRL	read / write app key
\$D9	CONTROL	enable CONFIG boot
\$D6	CONTROL	set boot mode
\$D5 / \$D4	CONTROL	enable / disable device
\$D1	STATUS	device enable status
\$D8	CONTROL	copy file
\$BB	STATUS	generate GUID
\$C1	STATUS	free heap (debug)
\$53	STATUS	Fuji status block
\$C8 \$C7 \$C6 \$C5 \$C2	CTRL/STAT	hash: input, compute, len, out, clear
\$BC \$BD \$BE \$BF	CTRL/STAT	QR: input, encode, length, output

Clock Device (type \$13)

Code	Call	Operation
\$54 'T'	STATUS	time, simple binary (7 bytes)
\$50 'P'	STATUS	time, ProDOS format
\$41 'A'	STATUS	time, APETIME binary
\$49 'I'	STATUS	time, ISO string (local)
\$5A 'Z'	STATUS	time, ISO string (UTC)
\$53 'S'	STATUS	time, Apple /// SOS format
\$47 'G'	STATUS	get time-zone string
\$54 'T'	CONTROL	set system time zone
\$74 't'	CONTROL	set alternate time zone

Send a time-format letter in lower case to read using the alternate zone instead of the system zone.

netcat in 6502

Here is the program promised on the title page: a *netcat*. It opens a raw TCP connection, then pumps bytes both ways — whatever the far end sends is printed to the screen, and whatever you type is sent to the far end — until the connection drops or you press **ESC**. It is the whole book in one listing: **SPINIT** finds the bus, **FINDNET** finds the device, and the five Network routines from Chapter 3 do the work.

What It Needs

The listing below is the **main program and data only**. Assemble it together with the routines already built in this book — they are listed once here so the program is complete:

- **SPINIT** (Listing 1-1) and the call primitives **SPSTAT**, **SPCTRL**, **SPREAD**, **SPWRITE** with **PAYLOAD**, **SPUNIT**, **SPCNT** (Listing 1-3);
- **FINDDEV** and **FINDNET** (Listings 2-1, 2-2);
- **NETCHAN**, **NETOPEN/COPYSPEC**, **NETCLOSE**, **NETSTAT**, **NETREAD**, **NETWRITE** and their variables **BW**, **CONN**, **NERR**, **MODE**, **TRANS**, **URLPTR** (Listings 3-1 through 3-6).

Change the address in **HOST** to the server you want to reach, set the assembler's origin, and run.

The Program

Listing C-1. FujiNet netcat — main loop and data

```
; =====
; FUJINET NETCAT for the Apple II          (ca65 syntax)
; socket <-> screen + keyboard, ESC to quit
; =====
KBD      =    $C000          ; keyboard data + ready (bit 7)
STROBE   =    $C010          ; clear keyboard strobe
COUT     =    $FD0D          ; monitor: print A (high bit set)
CROUT    =    $FD0E          ; monitor: print a carriage return
;
;          .ORG $2000
;
NETCAT   JSR    SPINIT        ; 1. find the SmartPort bus
        BCS    NODEV
        JSR    FINDNET       ; 2. find the Network device -> NETID
        LDA    NETID
        BEQ    NODEV
        LDA    #1
        JSR    NETCHAN       ; 3. operate on channel N1:
```

```

;
    LDA    #<HOST          ; 4. open TCP, read/write, no translation
    STA    URLPTR
    LDA    #>HOST
    STA    URLPTR+1
    LDA    #$0C             ; mode = read/write
    STA    MODE
    LDA    #$00             ; trans = none (binary)
    STA    TRANS
    JSR    NETOPEN
    BCS    OPENERR
;
; ---- the pump: drain the socket, then check the keyboard ----
PUMP    JSR    NETSTAT      ; how much is waiting? still connected?
        LDA    CONN
        BEQ    CLOSED      ; far end hung up
        LDA    NERR
        CMP    #136         ; EOF from the resource
        BEQ    CLOSED
        LDA    BW
        ORA    BW+1
        BEQ    KEYS        ; nothing waiting -> service keyboard
;
        LDA    BW+1         ; clamp the request to 512 bytes
        BEQ    RDOK        ; high byte 0 -> 255 or fewer, fine
        LDA    #$00
        STA    BW
        LDA    #$02
        STA    BW+1        ; force 512 (firmware returns <= waiting)
RDOK    JSR    NETREAD      ; pull the bytes into PAYLOAD
        LDX    #$00
EMIT    LDA    PAYLOAD,X
        ORA    #$80         ; set high bit for the Apple's video
        JSR    COUT
        INX
        CPX    SPCNT        ; SPCNT = bytes actually read
        BNE    EMIT
;
; ---- one key per pass, so the screen stays responsive -----
KEYS    LDA    KBD
        BPL    PUMP        ; no key down -> keep pumping
        STA    STROBE      ; acknowledge the key
        AND    #$7F        ; strip the ready bit
        CMP    #$1B        ; ESC?
        BEQ    QUIT
        STA    PAYLOAD     ; send this one byte
        LDA    #$01
        LDX    #$00
        JSR    NETWRITE
        JMP    PUMP
;
CLOSED  JSR    CROUT
        LDX    #$00
CMMSG   LDA    BYEMSG,X     ; "** CONNECTION CLOSED"
        BEQ    QUIT
        JSR    COUT
        INX

```

```

QUIT      BNE  CMSG
          JSR  NETCLOSE      ; tidy up and return to the monitor
          RTS

;
OPENERR   LDX  #$00
OMSG      LDA  OPMSG,X
          BEQ  ODONE
          JSR  COUT
          INX
          BNE  OMSG
ODONE     RTS
NODEV     LDX  #$00
NMSG      LDA  NOMSG,X
          BEQ  NDONE
          JSR  COUT
          INX
          BNE  NMSG
NDONE     RTS
;
; ----- data -----
HOST      .BYTE "N1:TCP://192.168.1.5:9000/", $00
BYEMSG    .BYTE $80,** CONNECTION CLOSED",$80,$00
OPMSG     .BYTE $80,** COULD NOT OPEN",$80,$00
NOMSG     .BYTE $80,** NO FUJINET FOUND",$80,$00

```

Trying It

On the other end, anything that speaks TCP will do. The classic test is the Unix `netcat` itself, listening on the port you named:

```

1 BRUN NETCAT

HELLO FROM THE APPLE //
and hello back from your laptop
the quick brown fox jumped over

** CONNECTION CLOSED
1

```

Type, and your keystrokes cross the room — or the world — and the reply paints onto a 1 MHz machine that predates the network it just joined. That is the whole trick of the FujiNet, and now it is yours to program: find the SmartPort, address the device, mind the length header, and the rest is just 6502.